RACIOCÍNIO AUTOMÁTICO PARA OPTIMIZAÇÃO

Luís Morgado

ISEL-DEI

PROBLEMAS DE OPTIMIZAÇÃO

Problemas em que se pretende encontrar a melhor configuração de parâmetros de modo a maximizar uma função de avaliação

- **Objectivo**:
  - Maximizar uma função de *valor* ou de *adequação*
  - Minimizar uma função de *perda* ou de *custo*
- **Resultado:**
  - Configuração de parâmetros (*estado*)

OPTIMIZAÇÃO

- **Selecção da melhor opção a partir de um conjunto de opções possíveis** de acordo com um critério de avaliação de opções
  - Maximização
  - Minimização
- **Critério de avaliação**
  - Medida de ganho
  - Medida de perda (*loss*)
  - Medida de adequação (*fitness*)

OPTIMIZAÇÃO

- **Métodos computacionais de optimização**
  - **Optimização matemática**
    - Exemplos
      - Optimização linear
      - Descida de gradiente
  - **Meta-heurísticas**
    - Formas de procura em espaços de estados
    - Exemplos
      - Procura local sôfrega (*Hill-climbing*)
      - Têmpera simulada (*Simulated anealing*)
      - Algoritmos genéticos

Exemplo: Optimização de Estruturas

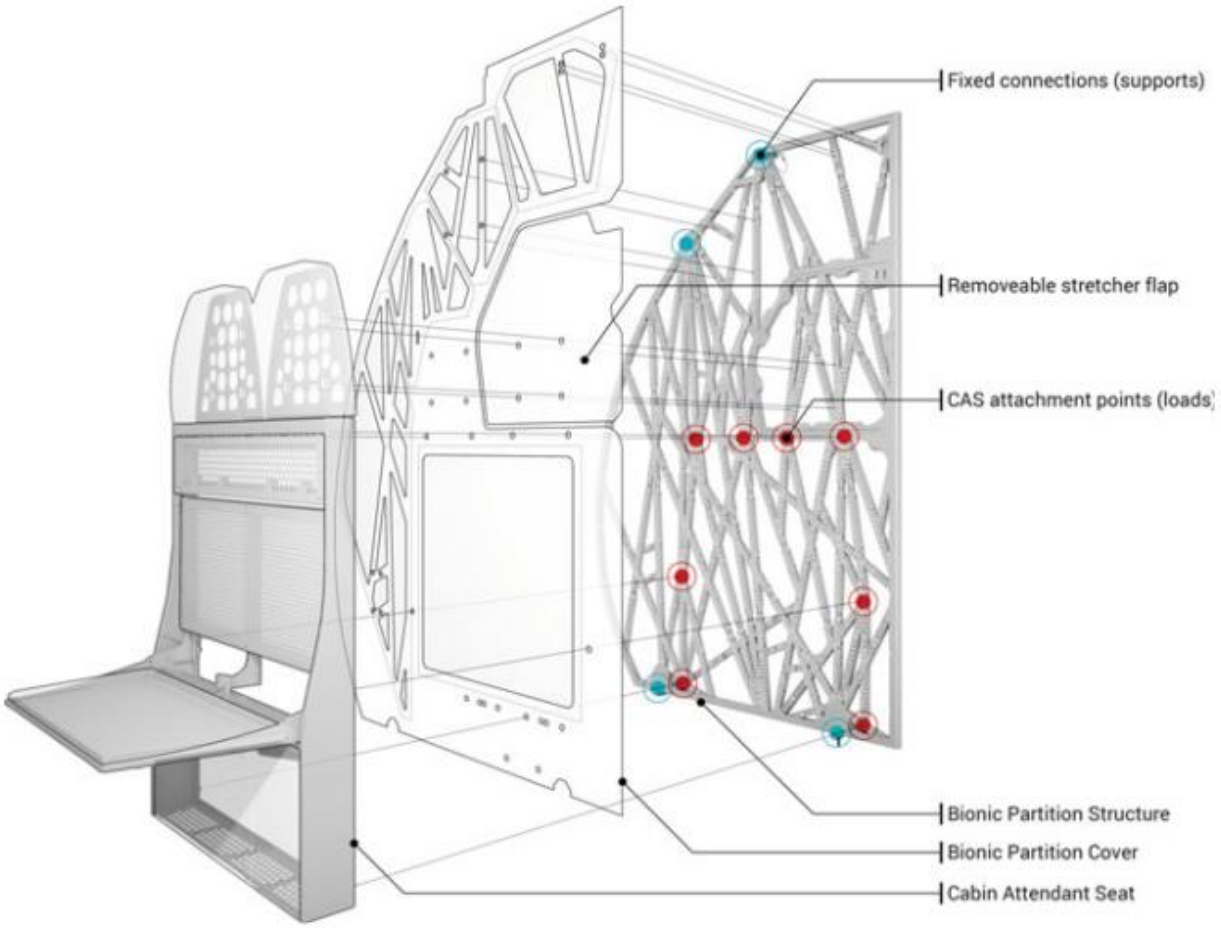


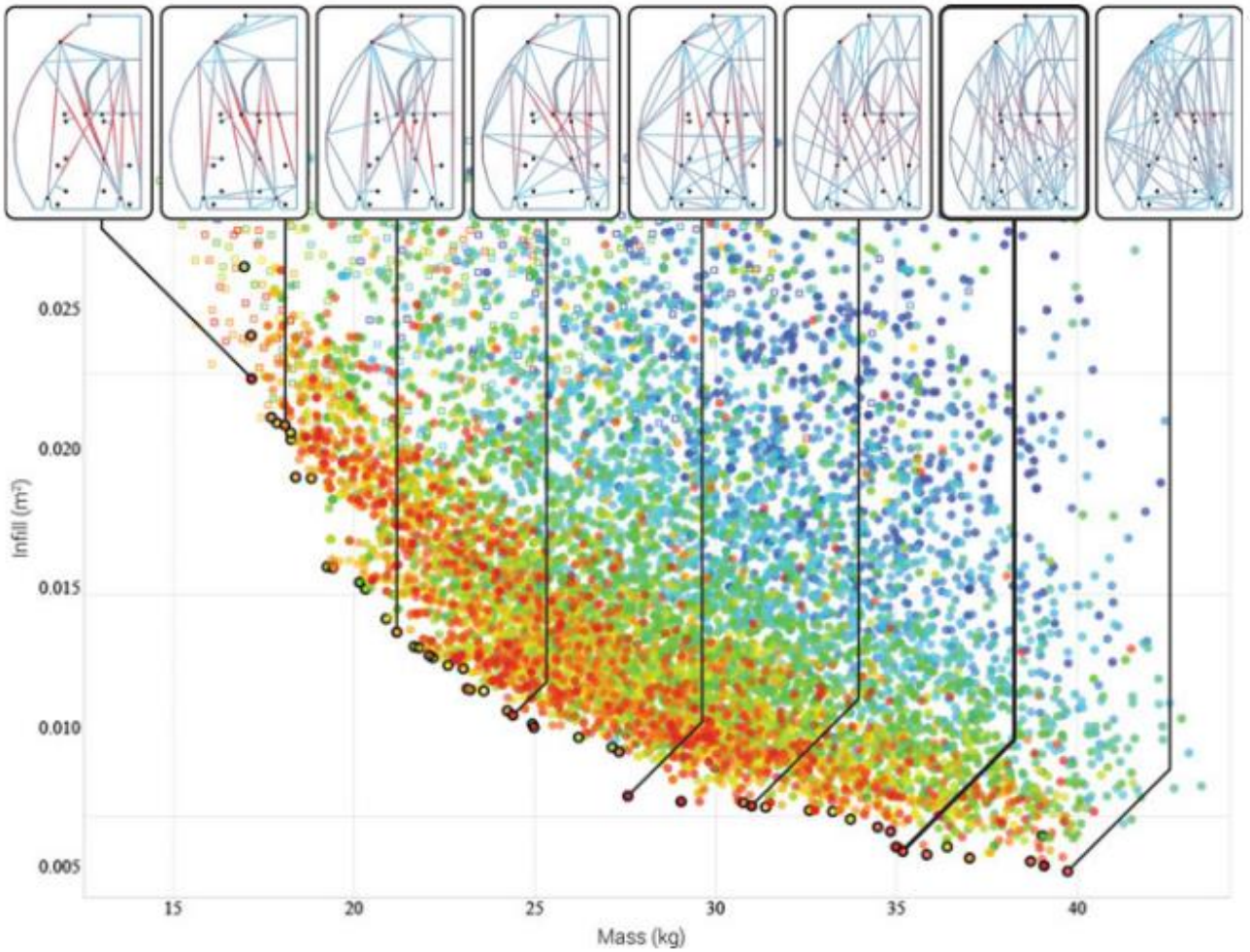
Fig. 1. Description of aircraft cabin partition design problem

[Nagy *et al.*, 2018]

Exemplo: Optimização de Estruturas

Exemplos de soluções possíveis

Dimensões (*parâmetros*) de optimização

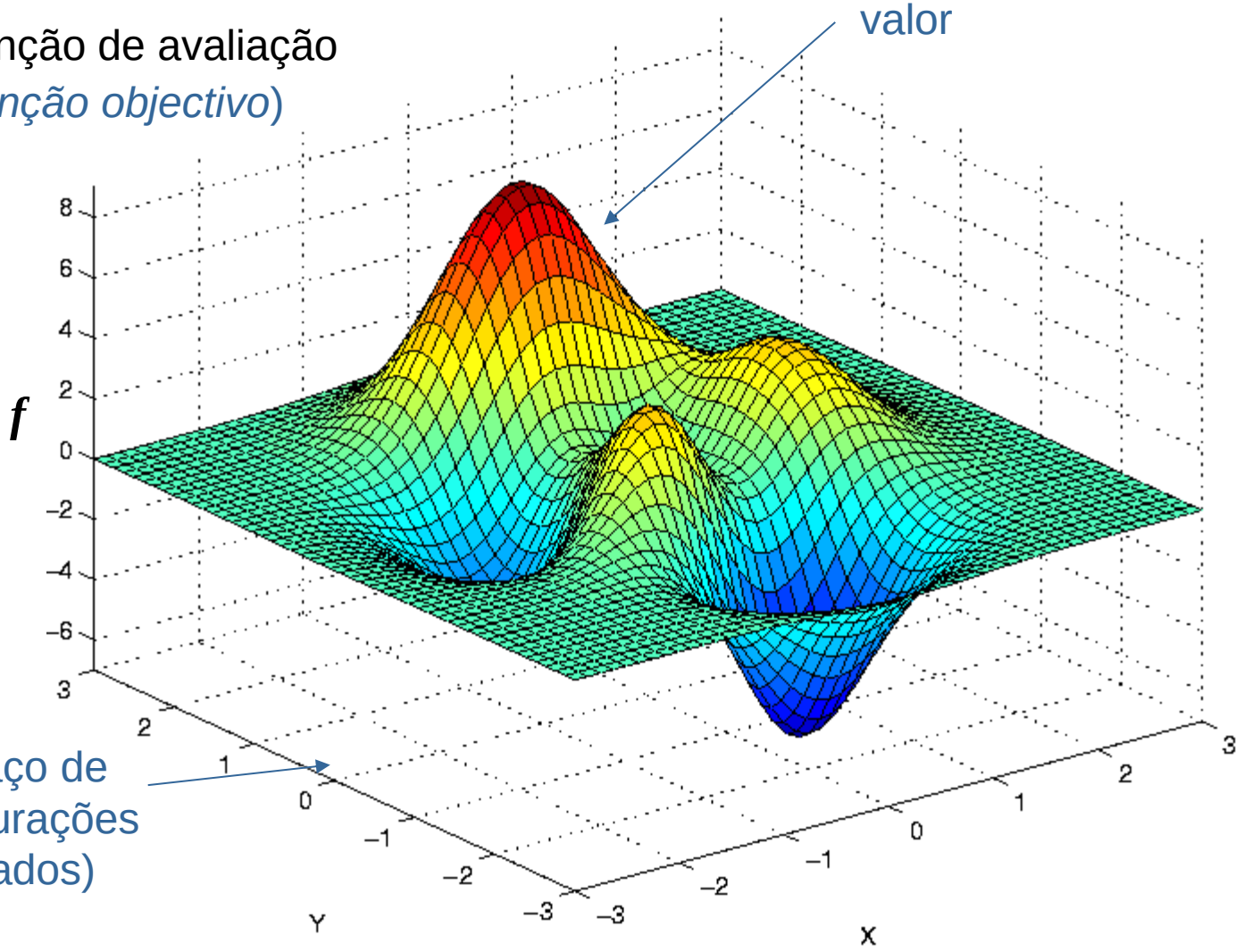


[Nagy *et al.*, 2018]

OPTIMIZAÇÃO

CRITÉRIO DE AVALIAÇÃO

f – Função de avaliação
(*função objectivo*)

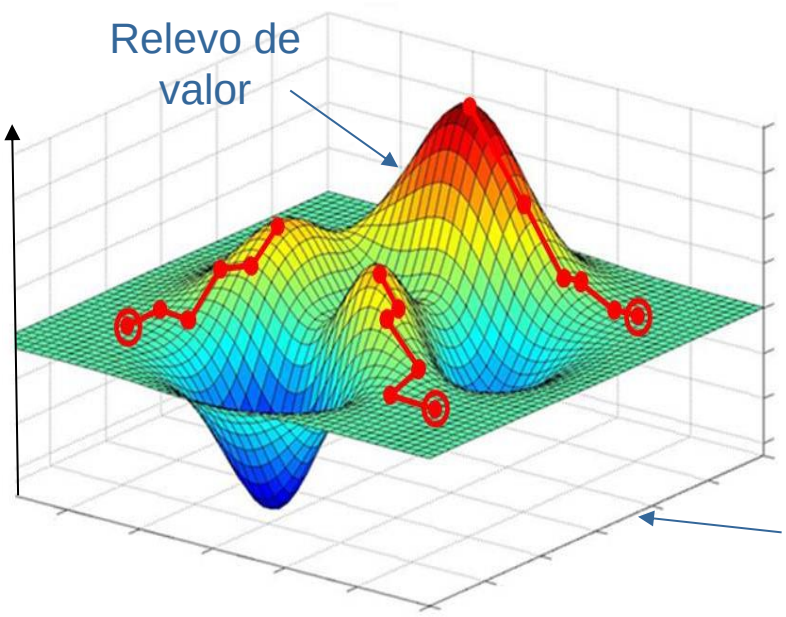


RACIOCÍNIO AUTOMÁTICO

RESOLUÇÃO DE PROBLEMAS DE OPTIMIZAÇÃO

- Saber qual a melhor configuração de parâmetros de modo a maximizar uma função de valor/adequação, ou a minimizar uma função de perda/custo
- **Resultado**: Configuração de parâmetros (estado)

f – Função de avaliação
(*função objectivo*)



Solução obtida por procura no espaço de estados do problema

Espaços de estados de grandes dimensões

↓

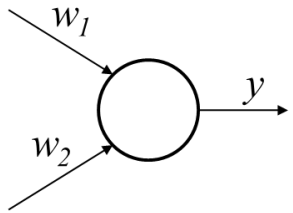
Procura exaustiva não é viável → Procura local

Exemplo: Redes neuronais artificiais

Representação de estado

- Parâmetros da rede

Exemplo:



Configuração: $\boldsymbol{w} = (w_1, w_2)$

↓

Estado

Vector de transformação de estado:

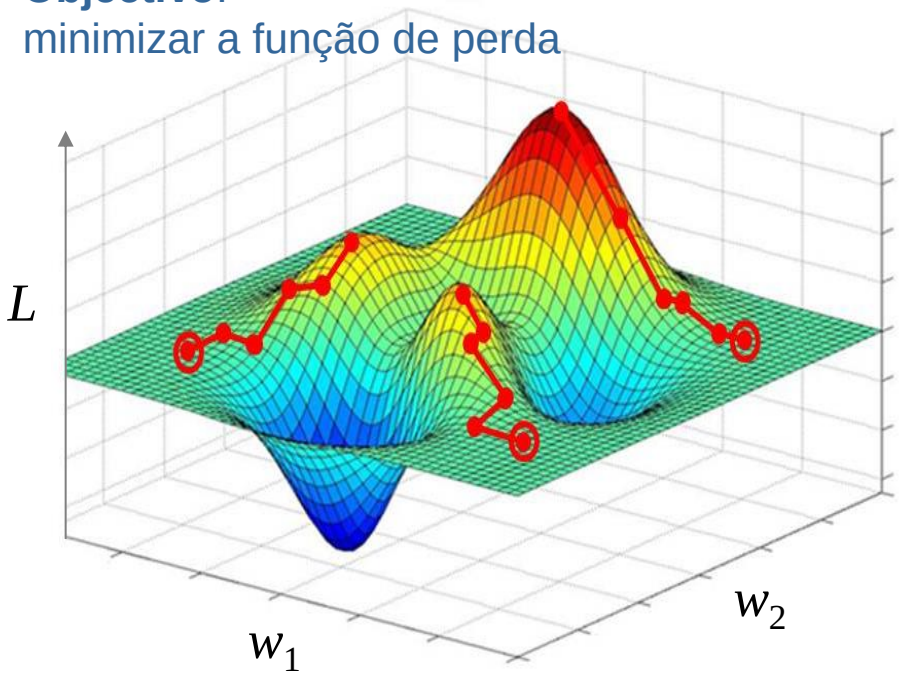
$$\Delta \boldsymbol{w} = (\Delta w_1, \Delta w_2)$$

Critério de avaliação

- Função de perda, por exemplo, definida pelo *Erro Quadrático Médio* (EQM)

$$L(\boldsymbol{y}, \boldsymbol{y}^N) = EQM(\boldsymbol{y}^N) = \frac{1}{K}\sum_{k=1}^{K}(y_k^N - y_k)^2$$

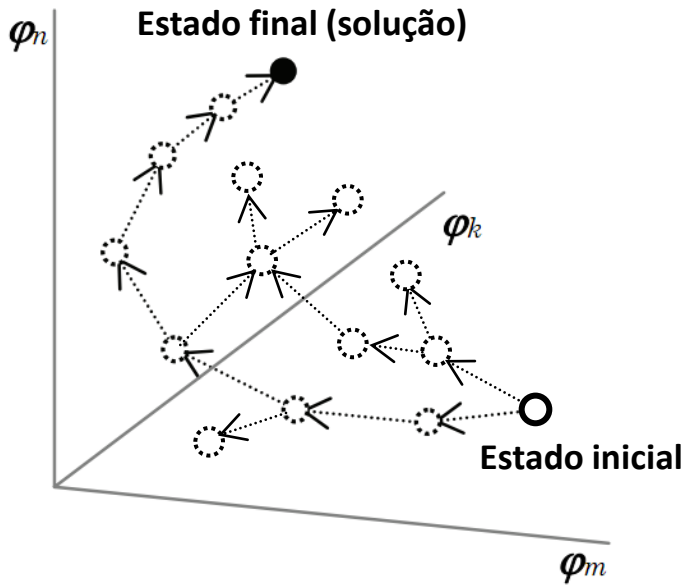
Objectivo:
minimizar a função de perda



RACIOCÍNIO ATRAVÉS DE PROCURA

RESOLUÇÃO DE PROBLEMAS DE OPTIMIZAÇÃO

- **Estado**
  - Representa uma **configuração**
  - **Espaço de estados**
- **Transição**
  - Representa uma **transformação de estado**, que pode ser representada de várias formas:
    - **Operador** (de transição de estado)
    - **Vector** (de transição de estado)
    - **Sucessor** (estado seguinte da transição)
- **Valor**
  - Função de valor de estado
- **Solução**
  - Estado final



OPTIMIZAÇÃO ATRAVÉS DE PROCURA

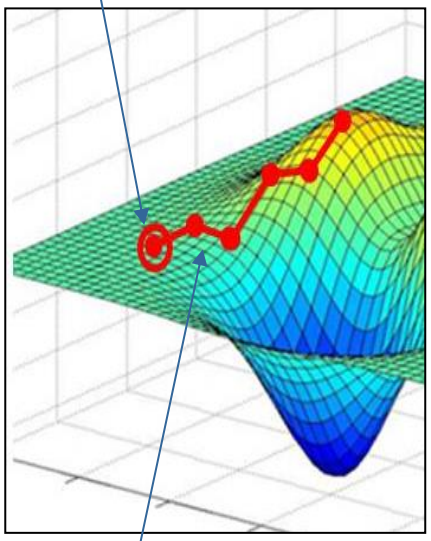
MÉTODOS DE PROCURA LOCAL

- Métodos de procura em espaços de estados utilizados em problemas de optimização, em que **a procura é realizada por exploração de estados adjacentes ao estado actual** (*estados sucessores*) , em vez da exploração sistemática do espaço de estados de forma global
- Utilizados em problemas onde o estado (solução) óptimo pode ser encontrado através de **transformações incrementais do estado actual**

OPTIMIZAÇÃO ATRAVÉS DE PROCURA

MÉTODOS DE PROCURA LOCAL

- **A procura tem início** com um *estado inicial*
- **Por iterações sucessivas**, são gerados e avaliados os *estados sucessores* do estado actual
  - Estados obtidos por transformações de estado incrementais
- **Em cada iteração** é seleccionado um estado sucessor como novo estado actual
  - Na sua forma mais simples, é seleccionado o estado sucessor com melhor avaliação
- **Este processo é repetido** até que não existam estados sucessores com melhor avaliação que o estado actual (ou até a satisfação de um critério de finalização definido)



PROCURA EM ESPAÇOS DE ESTADOS LOCAL

ALGORITMO *HILL-CLIMBING*

No algoritmo *Hill-Climbing*, a cada passo da procura o nó actual é substituído pelo melhor nó sucessor, ou seja, o nó com maior valor

- Se for utilizada uma avaliação de custo, em vez da maximização do valor deverá ser realizada a minimização do custo

Algoritmo *Hill-Climbing* geral:

```
function HILL-CLIMBING(problem) returns a state that is a local maximum

    current ← MAKE-NODE(problem.INITIAL-STATE)
    loop do
        neighbor ← a highest-valued successor of current
        if neighbor.VALUE ≤ current.VALUE then return current.STATE
        current ← neighbor
```

[Russel & Norvig, 2010]

PROCURA EM ESPAÇOS DE ESTADOS LOCAL

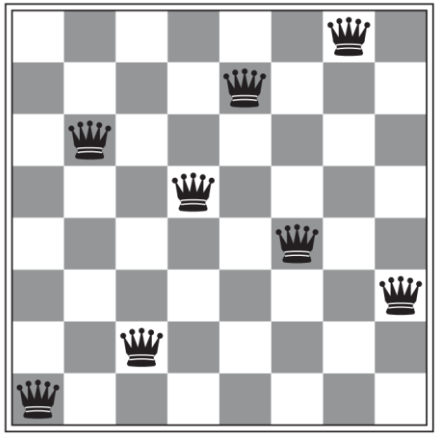
Algoritmo *Hill-Climbing* detalhado

```
1. function hill_climbing(problema): Estado
2.     estado = problema.estado_inicial
3.     valor_estado = -∞
4.     while true:
5.         sucessores = problema.sucessores(estado)
6.         novo_estado = max(sucessores, problema.avaliar)
7.         valor_novo_estado = problema.avaliar(novo_estado)
8.         if novo_estado is none or
9.             valor_novo_estado <= valor_estado:
10.            break
11.        else:
12.            estado = novo_estado
13.            valor_estado = valor_novo_estado
14.    return estado
```

ALGORITMO *HILL-CLIMBING*

Exemplo: Problema das rainhas

Como dispor um conjunto de rainhas num tabuleiro de xadrez minimizando o número de conflitos (rainhas atacadas)



Representação de estado

Configuração do tabuleiro

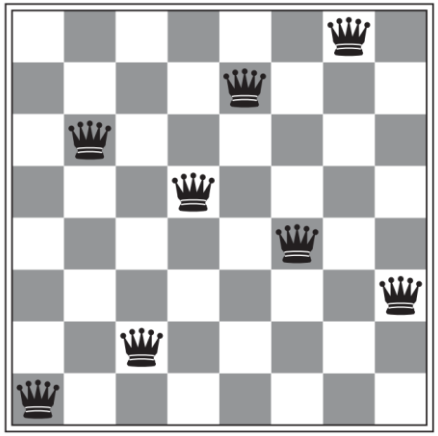
```
. . . . . . R .
. . . . R . . .
. R . . . . . .
. . . R . . . .
. . . . . R . .
. . . . . . . R
. . R . . . . .
R . . . . . . .
```

R - Rainha

ALGORITMO *HILL-CLIMBING*

Exemplo: Problema das rainhas

Como dispor um conjunto de rainhas num tabuleiro de xadrez minimizando o número de conflitos (rainhas atacadas)



Representação de estado

Configuração do tabuleiro

Número de estados possíveis: $\frac{(8^2)!}{(8^2-8)!} = 178{,}462{,}987{,}637{,}760$

```
. . . . . . R .
. . . . R . . .
. R . . . . . .
. . . R . . . .
. . . . . R . .
. . . . . . . R
. . R . . . . .
R . . . . . . .
```

Problemas:

- Redundância de informação
- Processamento ineficiente

R - Rainha

Exemplo: Problema das rainhas

Representação de estado

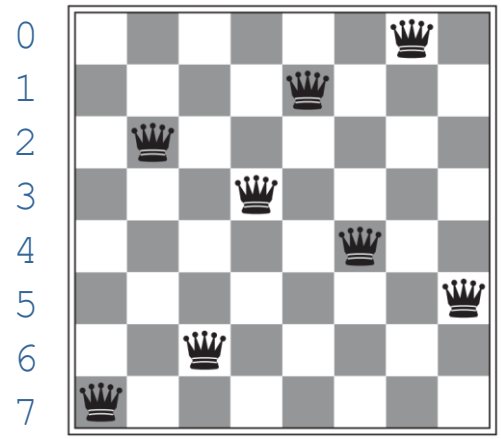
Uma representação de estado adequada deve expressar as restrições que caracterizam o problema

- **Reduz a dimensionalidade do espaço de estados**
- **Torna a procura mais eficiente**

Representação de uma rainha por coluna

- Expressa a restrição de não colocar mais que uma rainha por coluna para que não existam conflitos nas colunas

Número de linha



Representação de estado

7	2	6	3	1	4	0	5

Número de estados possíveis: $8^8 = 16{,}777{,}216$

Vantagens:

- Eliminação de redundância
- Processamento eficiente

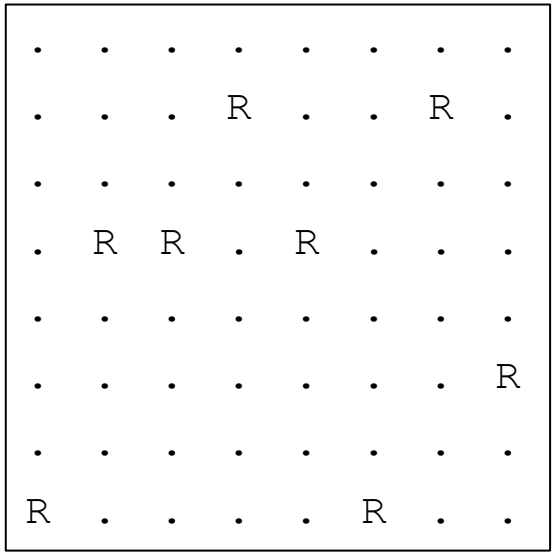
ALGORITMO *HILL-CLIMBING*

Exemplo: Problema das rainhas

Iteração 1: Estado actual = Estado inicial

`[7, 3, 3, 1, 3, 7, 1, 5]`

Configuração correspondente



Estados sucessores

Avaliação (perda, definida pelo número de conflitos)

```
[0, 3, 3, 1, 3, 7, 1, 5]: -9
[1, 3, 3, 1, 3, 7, 1, 5]: -12
[2, 3, 3, 1, 3, 7, 1, 5]: -11
[3, 3, 3, 1, 3, 7, 1, 5]: -12
[4, 3, 3, 1, 3, 7, 1, 5]: -11
...
```

Melhor sucessor → Estado actual

`[7, 3, 3, 1, 4, 7, 1, 5]: -8`

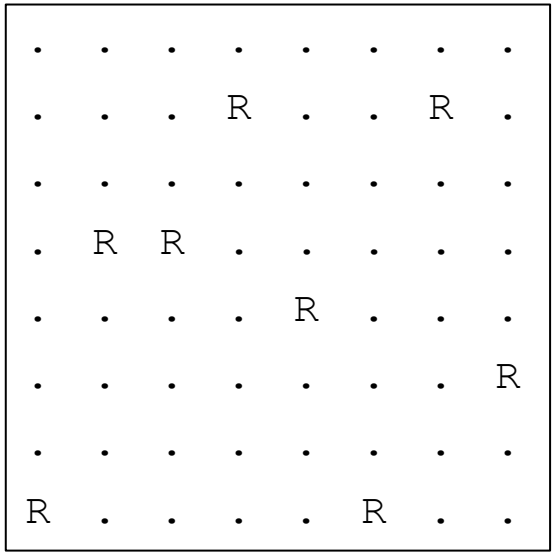
ALGORITMO *HILL-CLIMBING*

Exemplo: Problema das rainhas

Iteração 2: Estado actual

```
[7, 3, 3, 1, 4, 7, 1, 5]
```

Configuração correspondente



Estados sucessores

Avaliação (perda, definida pelo número de conflitos)

```
[0, 3, 3, 1, 4, 7, 1, 5]: -7
[1, 3, 3, 1, 4, 7, 1, 5]: -9
[2, 3, 3, 1, 4, 7, 1, 5]: -8
[3, 3, 3, 1, 4, 7, 1, 5]: -8
[4, 3, 3, 1, 4, 7, 1, 5]: -9
...
```

Melhor sucessor → Estado actual

```
[7, 0, 3, 1, 4, 7, 1, 5]: -5
```

ALGORITMO *HILL-CLIMBING*

Características

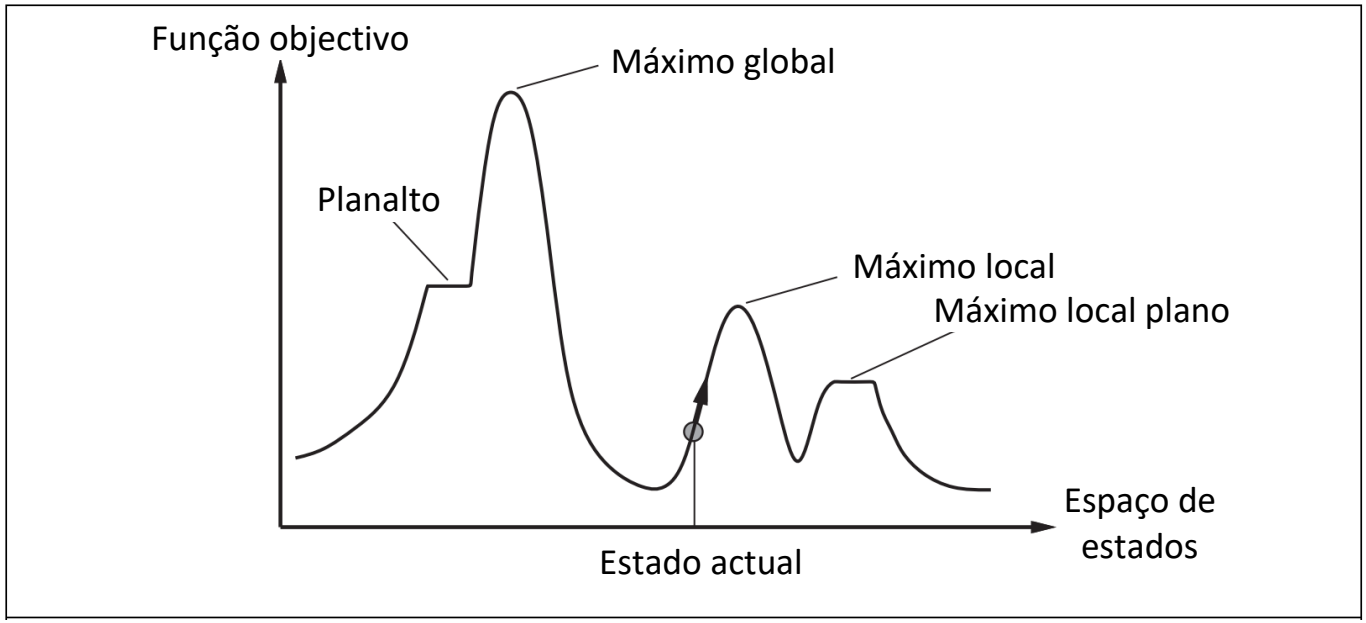
- **Completo**
  - Não é completo
- **Óptimo**
  - Não é óptimo
- **Complexidade**
  - **Espacial**
    - Linear, em função do factor de ramificação ***b***
    - **O(*b*)**
  - **Temporal**
    - Linear, em função da dimensão da solução ***d***
    - **O(*d*)**

ALGORITMO *HILL-CLIMBING*

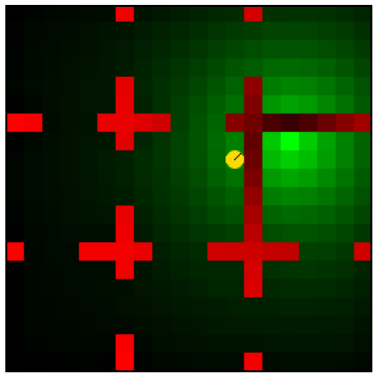
PROBLEMA: Óptimos Locais

Exemplo: Espaço de estados unidimensional

O relevo corresponde à *função objectivo*, da qual se pretende encontrar o máximo global



Adaptado de [Russel & Norvig, 2010]



Exemplo: Espaço de estados bidimensional

A cor representa o valor da função objectivo

PROCURA EM ESPAÇOS DE ESTADOS LOCAL

Soluções para o problema dos óptimos locais

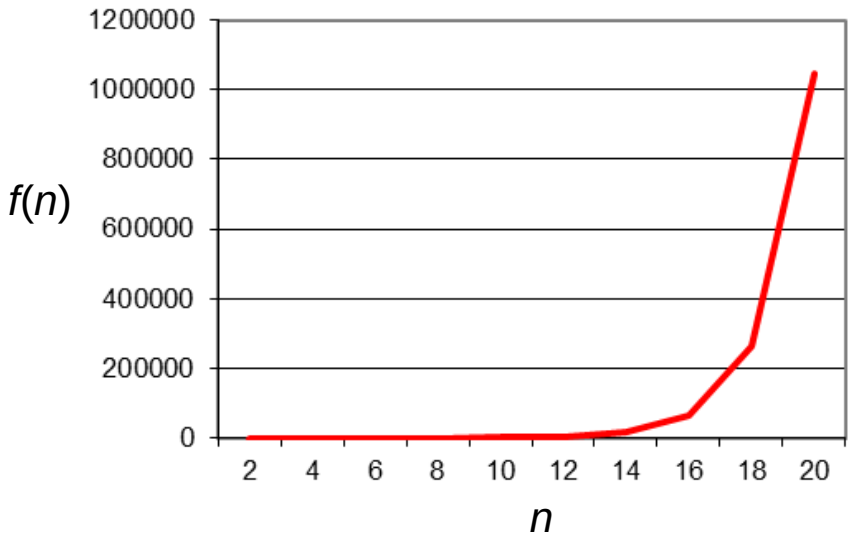
- **Manter memória de estados visitados**
  - **Complexidade espacial exponencial**
    - Não é viável para problemas gerais com espaços de estados de grande dimensionalidade

Crescimento exponencial:

$f(n) = O(m^n)$

n – dimensão do problema

Exemplo: $f(n) = 2^n$



PROCURA EM ESPAÇOS DE ESTADOS LOCAL

Soluções para o problema dos óptimos locais

- **Manter memória de estados visitados**
  - **Complexidade espacial exponencial**

Exemplo:

Problema das 8 rainhas com representação de uma rainha por coluna

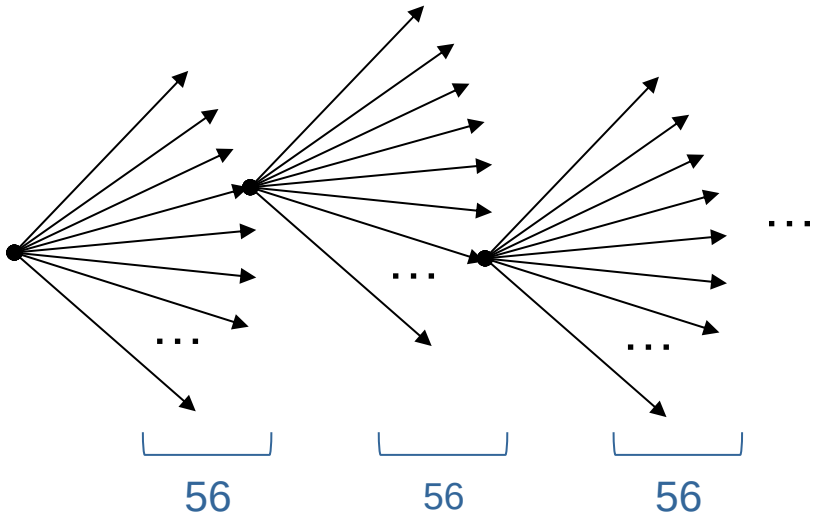
Complexidade espacial com memória de estados visitados: $C = b^n$

Factor de ramificação:

$b = 8 \times 7 = 56 \rightarrow C = 56^n$

Número de colunas

Número de movimentos por coluna



Dimensão do problema $n = 8$

PROCURA EM ESPAÇOS DE ESTADOS LOCAL

Soluções para o problema dos óptimos locais

- **Mecanismos de exploração não exaustiva**
  - Amostragem estocástica do espaço de estados
  - Controlo de *exploração / aproveitamento*
    - **Exploração**
      - Seleccionar operador com um critério de base aleatória
    - **Aproveitamento**
      - Seleccionar transição que maximiza valor local
        - » **Opção sôfrega (*greedy*)**

PROCURA EM ESPAÇOS DE ESTADOS LOCAL

Soluções para o problema dos óptimos locais

- **Modos de procura**
  - **Modo local**
    - Exploração a partir do estado actual (guiada)
    - **Exemplo**:
      - Métodos de *têmpera simulada* (*simulated annealing*)
  - **Modo global**
    - Exploração independente do estado actual (amostragem)
    - **Exemplo**:
      - *Hill-Climbing* com reinício aleatório

ALGORITMO *HILL-CLIMBING*

Variantes

- ***Hill-Climbing* estocástico**
  - **Escolha aleatória entre sucessores que aumentam o valor de estado**
  - Convergência mais lenta que *Hill-climbing*
  - Pode encontrar melhores soluções
    - Consoante a topologia do espaço de estados

ALGORITMO *HILL-CLIMBING*

Variantes

- ***Hill-Climbing* estocástico com único sucessor**
  - **Sucessores gerados aleatoriamente até um aumentar o valor de estado**
  - Útil quando o número de sucessores por estado é muito elevado

ALGORITMO *HILL-CLIMBING*

Variantes

- ***Hill-Climbing* com reinício aleatório**
  - **Procura é reiniciada a partir de estados iniciais aleatórios**, até ser atingido um objectivo ou ser atingido um número máximo de reinícios
  - Exploração independente do estado actual (*amostragem global*)
  - Pode encontrar melhores soluções
    - Dependente do número de reinícios

ALGORITMO *HILL-CLIMBING*

Algoritmo *Hill-Climbing* com reinício aleatório

```
1. function random_restart_hill_climbing(problema,
2.                                        max_reinicios)
3.     melhor_estado = none
4.     valor_melhor_estado = -∞
5.     for reinicio = 1 to max_reinicios:
6.         estado = hill_climbing(problema)
7.         if problema.solucao(estado):
8.             melhor_estado = estado
9.             break
10.        else:
11.             valor_estado = problema.avaliar(estado)
12.             if valor_estado > valor_melhor_estado:
13.                 melhor_estado = estado
14.                 valor_melhor_estado = valor_estado
15.    return melhor_estado
```

REFERÊNCIAS

[Russel & Norvig, 2010]

S. Russell, P. Norvig, “Artificial Intelligence: A Modern Approach”, 3rd Ed., Prentice Hall, 2010

[Beire *et. Al.*, 2014]

A. Beire, N. Cota, H. Pita, A. Rodrigues, “Automatic tuning of Okumura-Hata model on railway communications”, International Symposium on Wireless Personal Multimedia Communications (WPMC), 2014

[Nagy *et al.*, 2018]

D. Nagy, D. Zhao, D. Benjamin, “Nature-Based Hybrid Computational Geometry System for Optimizing Component Structure”, Humanizing Digital Reality, Springer Nature, 2018